

SQL - Funkcje okna (Window functions) Lab 2

Imiona i nazwiska: Mateusz Lampert, Marek Małek

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu “tradycyjnych” konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DDataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020
- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>

- <https://www.sqlservertutorial.net/sql-server-window-functions/>
- <https://www.sqlshack.com/use-window-functions-sql-server/>
- <https://www.postgresql.org/docs/current/tutorial-window.html>
- <https://www.postgresqltutorial.com/postgresql-window-function/>
- <https://www.sqlite.org/windowfunctions.html>
- <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze “zmodyfikowanej wersji” bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, row_number(), rank(), dense_rank()

```
select productid, productname, unitprice, categoryid,
       row_number() over(partition by categoryid order by unitprice desc) as rowno,
       rank() over(partition by categoryid order by unitprice desc) as rankprice,
       dense_rank() over(partition by categoryid order by unitprice desc) as
denserankprice
from products;
```

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje row_number(), rank(), dense_rank(). Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

Wynik zapytania z polecenia:

productid	productname	unitprice	categoryid	rowno	rankprice	denserankprice
38	Côte de Blaye	263.5	1	1	1	1
43	Iphoh Coffee	46	1	2	2	2
2	Chang	19	1	3	3	3
1	Chai	18	1	4	4	4
76	Lakkalikööri	18	1	5	4	4
35	Steeleye Stout	18	1	6	4	4
39	Chartreuse verte	18	1	7	4	4
70	Outback Lager	15	1	8	8	5
34	Sasquatch Ale	14	1	9	9	6
67	Laughing Lumberjack Lager	14	1	10	9	6
75	Rhönbräu Klosterbier	7.75	1	11	11	7
24	Guananã Fantástica	4.5	1	12	12	8
63	Veggie-spread	43.9	2	1	1	1

Figure 1: alt text

Komentarz:

- row_number to numer danego wiersza wewnątrz okna, zgodnie z zadaną kolejnością (tutaj: unitprice desc, natomiast zgodnie z dokumentacją oracle, funkcja row_number nie jest deterministyczna w przypadku remisów). W przypadku row_number nie ma remisów w wartościach, zawsze dostajemy wartości 1..*ilosc wierszy w oknie*.
- rank to ranga danego wiersza wewnątrz okna, zgodnie z zadaną kolejnością (tutaj: unitprice desc). W przypadku remisów, wszystkie wiersze o tej samej wartości dostają tę samą rangę r, natomiast wiersze o następnej w kolejności wartości dostają rangę r+k, gdzie k-ilość remisujących wierszy. rank zawsze przyjmowało wartości 1..*ilosc wierszy w oknie*.
- dense_rank działa podobnie jak rank, natomiast w przypadku remisów, wszystkie wiersze o tej samej wartości dostają tę samą rangę r, a wiersze o następnej w kolejności wartości dostają rangę r+1 (nie przeskakujemy wartości).

Te same wyniki można także uzyskać nie korzystając z funkcji okna, z wykorzystaniem podzapytań lub instrukcji join:

- row_number:

```
-- row_number() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid,
count(p2.productid) + 1 as rowno_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and
           (p2.unitprice > p1.unitprice or (p2.unitprice = p1.unitprice and
p2.productid < p1.productid))
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, rowno_custom;
```

```
-- row_number() z subquery
select productid, productname, unitprice, categoryid,
(
    select count(*) + 1
    from products p2
    where p2.categoryid = p1.categoryid
    and (
        p2.unitprice > p1.unitprice
        or (p2.unitprice = p1.unitprice and p2.productid < p1.productid)
    )
) as rowno_custom
from products p1
order by p1.categoryid, rowno_custom;
```

- rank:

```
-- rank() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid,
count(p2.productid) + 1 as rankprice_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and p2.unitprice >
p1.unitprice
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, rankprice_custom;
```

```
-- rank() z subquery
select productid, productname, unitprice, categoryid,
(
    select count(*) + 1
    from products p2
    where p2.unitprice > p1.unitprice and p2.categoryid = p1.categoryid
) as rankprice_custom
from products p1
order by p1.categoryid, rankprice_custom;
```

- dense_rank:

```
-- dense_rank() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid, count(distinct
p2.unitprice) + 1 as denserankprice_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and p2.unitprice >
p1.unitprice
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, denserankprice_custom;
```

```
-- dense_rank() z subquery
select productid, productname, unitprice, categoryid,
(
```

```

        select count(distinct p2.unitprice) + 1
        from products p2
        where p2.unitprice > p1.unitprice and p2.categoryid = p1.categoryid
    ) as denserankprice_custom
from products p1
order by p1.categoryid, denserankprice_custom;

```

W celu porównania wyników uruchamiamy zapytanie łączące wszystkie te zapytania:

```

with custom_rn as (select p1.productid,
                        count(p2.productid) + 1 as rowno_custom
                    from products p1
                    left join products p2
                        on p2.categoryid = p1.categoryid and
                           (p2.unitprice > p1.unitprice or
                              (p2.unitprice = p1.unitprice and
p2.productid < p1.productid))
                    group by p1.productid),
    custom_rk as (select p1.productid,
                        count(p2.productid) + 1 as rankprice_custom
                    from products p1
                    left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice
                    group by p1.productid),
    custom_dk as (select p1.productid,
                        count(distinct p2.unitprice) + 1 as
denserankprice_custom
                    from products p1
                    left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice
                    group by p1.productid)
select p.productid,
       p.productname,
       p.unitprice,
       p.categoryid,
       custom_rn.rowno_custom,
       row_number() over (partition by categoryid order by p.unitprice desc,
p.productid) as rowno,
       custom_rk.rankprice_custom,
       rank() over (partition by categoryid order by p.unitprice desc)
as rankprice,
       custom_dk.denserankprice_custom,
       dense_rank() over (partition by categoryid order by p.unitprice desc)
as denserankprice
from products p
    join custom_rn on custom_rn.productid = p.productid
    join custom_rk on custom_rk.productid = p.productid

```

```
join custom_dk on custom_dk.productid = p.productid
order by p.categoryid, custom_rn.rowno_custom;
```

	productid	productname	unitprice	categoryid	rowno_custom	rowno	rankprice_custom	rankprice	denserankprice_custom	denserankprice
1	38	Cote de Boe	268.5	1	1	1	1	1	1	1
2	42	Tosh Coffee	46	1	2	2	2	2	2	2
3	2	Cheng	19	1	3	3	3	3	3	3
4	1	Chai	18	1	4	4	4	4	4	4
5	38	Steelage Stout	18	1	5	5	4	4	4	4
6	39	Chartrouse verte	18	1	6	6	4	4	4	4
7	76	LakkaLibbri	18	1	7	7	4	4	4	4
8	76	Outback Lager	15	1	8	8	8	8	5	5
9	34	Sasquatch Ale	14	1	9	9	9	9	6	6
10	67	Laughing Lumberjack Lager	14	1	10	10	9	9	6	6
11	75	Rhinoru Klostorbier	7.75	1	11	11	11	11	7	7
12	24	Guarana Fantastica	4.5	1	12	12	12	12	8	8
13	63	Veggie-spread	43.9	2	1	1	1	1	1	1
14	8	Northwoods Cranberry Sauce	40	2	2	2	2	2	2	2

Figure 2: alt text

Jak widać na załączonym zrzucie ekranu, wszystkie funkcje oraz nasze customowe odpowiedniki dają identyczne rezultaty.

Ze względu na różnicę w wydajności podzapytania względem inner-joina, w dalszej części konspektu będziemy korzystać z metody wykorzystującej inner-joina (porównanie wykonane dla silnika Postgres oraz funkcji row_number, wydajność pozostałych funkcji ma podobną charakterystykę):

- funkcja

okna:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
Select					
Transformation (WindowAgg)		77	5.92	4.21	Parallel Aware = false; Async Capable = ...
Sort		77	4.38	4.18	Parent Relationship = Outer; Parallel Aw...
Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel Aw...

- inner-join:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
Select					
Sort		77	25.08	24.88	Parallel Aware = false; Async Capable ...
Aggregate		77	22.47	21.51	Strategy = Hashed; Partial Mode = Sim...
Hash Join		274	20.14	2.73	Parent Relationship = Outer; Parallel A...
Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel A...
Transformation (Hash)		77	1.77	1.77	Parent Relationship = Inner; Parallel Aw...
Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel A...

- podzapytanie:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
Select					
Sort		77	201.5	201.3	Parallel Aware = false; Async Capable =...
Full Scan (Seq Scan)	table: products;	77	198.89	0.0	Parent Relationship = Outer; Parallel Aw...
Aggregate		1	2.58	2.55	Strategy = Plain; Partial Mode = Simple...
Full Scan (Seq Scan)	table: products;	3	2.54	0.0	Parent Relationship = Outer; Parallel Aw...

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wyników powinien zawierać:

- rok
- id produktu
- nazwę produktu
- cenę

- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

W przypadku zapytania wykorzystującego funkcję okna, korzystamy z funkcji `dense_rank`, a następnie wybieramy unikalne wartości (rok, `productid`, cena) z `denserankprice < 4` - w rezultacie uzyskamy 4 najwyższe ceny, bez względu na występujące remisy. Ze względu na długi czas wykonania `explain analyse` (w przypadku zapytania z `inner-join`, zapytanie z funkcją okna działa bardzo szybko), zbiór wynikowy został ograniczony jedynie do produktów o `productid < 10`.

```
-- zapytanie wykorzystujące dense_rank()

with t as (select distinct on (extract(year from date), productid, unitprice)
extract(year from date) as year,
productid,
productname,
unitprice,
date,
dense_rank() over (
partition by extract(year from date), productid
order by unitprice desc
)
as denserankprice
from product_history p)
select *
from t
where t.denserankprice <= 4 and t.productid < 10
order by year, productid, unitprice desc;

-- zapytanie bez funkcji okna (inner-join)

with t as (select distinct on (extract(year from date), productid, unitprice)
extract(year from date) as year,
```

```

productid,
productname,
unitprice,
date
        from product_history),
dr as (select pl.year,
            pl.productid,
            pl.productname,
            pl.unitprice,
            pl.date,
            count(distinct p2.unitprice) + 1 as denserankprice
        from t pl
        left join t p2 on p2.year = pl.year and p2.productid =
pl.productid and p2.unitprice > pl.unitprice
        group by pl.year, pl.productid, pl.productname, pl.unitprice,
pl.date)
select *
from dr
where denserankprice <= 4 and productid < 10
order by year, productid, unitprice desc;

```

Rezultaty:

• Z funkcją okna:

	year	productid	productname	unitprice	date	denserankprice	
1	1940	1	Chai	27.97	1940-01-05		1
2	1940	1	Chai	27.95	1940-11-26		2
3	1940	1	Chai	27.90	1940-11-29		3
4	1940	1	Chai	27.86	1940-10-09		4
5	1940	2	Chang	28.93	1940-03-09		1
6	1940	2	Chang	28.88	1940-11-07		2
7	1940	2	Chang	28.87	1940-05-26		3
8	1940	2	Chang	28.55	1940-11-21		4
9	1940	3	Aniseed Syrup	19.97	1940-06-20		1
10	1940	3	Aniseed Syrup	19.96	1940-12-06		2
11	1940	3	Aniseed Syrup	19.88	1940-01-14		3
12	1940	3	Aniseed Syrup	19.77	1940-05-26		4
13	1940	4	Chef Anton's Cajun Seasoning	31.97	1940-03-01		1

• Z inner-join:

1	1940	1	Chai	27.97	1940-01-05		1
2	1940	1	Chai	27.95	1940-11-26		2
3	1940	1	Chai	27.90	1940-11-15		3
4	1940	1	Chai	27.86	1940-10-09		4
5	1940	2	Chang	28.93	1940-03-09		1
6	1940	2	Chang	28.88	1940-11-07		2
7	1940	2	Chang	28.87	1940-05-26		3
8	1940	2	Chang	28.55	1940-11-21		4
9	1940	3	Aniseed Syrup	19.97	1940-06-20		1
10	1940	3	Aniseed Syrup	19.96	1940-12-06		2
11	1940	3	Aniseed Syrup	19.88	1940-07-28		3
12	1940	3	Aniseed Syrup	19.77	1940-05-26		4
13	1940	4	Chef Anton's Cajun Seasoning	31.97	1940-03-01		1

Zapytania dla MSSQL oraz SQLite są generalnie identyczne, z dokładnością do wyciągania roku z daty:

```
-- postgres
extract(year from date) as year
```

```
-- mssql
year(date) as year
```

```
-- sqlite
strftime('%Y', date) as year
```

Porównanie planów zapytań (Postgres):

- zapytanie dense_rank

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↳ Select								Planning = ("Shar...
↳ Unknown (Incremental Sort)		55417	2988.0	138711.88	474.498	86143.07	236.484	Parallel Aware = f...
↳ Access (Subquery Scan)		55417	2988.0	137204.15	473.435	86139.91	233.995	Parent Relationshi...
↳ Unique		166251	249180.0	135126.02	460.623	86139.91	233.924	Parent Relationshi...
↳ Unknown (Incremental Sort)		275891	270000.0	133056.83	437.155	86139.91	233.924	Parent Relationshi...
↳ Transformation (Window#		275891	270000.0	124479.14	352.849	86139.67	233.636	Parent Relationshi...
↳ Unknown (Gather Merge)		275891	270000.0	118271.59	284.779	86139.54	233.584	Parent Relationshi...
↳ Sort		114955	90000.0	85426.9	198.862	85139.51	192.912	Parent Relationshi...
↳ Full Scan (Seq Sc	table: product_history;	114955	90000.0	71154.64	115.058	0.0	7.88	Parent Relationshi...

- zapytanie inner-join

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↳ Select								Planning = ("Share...
↳ Unknown (Incremental Sort)		7566	2988.0	552492.36	25296.023	526300.58	1252.499	Parallel Aware = fa...
↳ Unique		231000	2077307.0	526265.15	1553.258	239902.58	956.573	Parent Relationshi...
↳ Unknown (Gather Merge)		2310000	2310000.0	508940.15	1333.916	239902.58	956.572	Parent Relationshi...
↳ Sort		962500	770000.0	241308.81	971.498	238902.56	923.968	Parent Relationshi...
↳ Full Scan (Seq Sc	table: product_history;	962500	770000.0	70867.25	181.0	0.0	85.046	Parent Relationshi...
↳ Aggregate		7566	2988.0	25991.86	25288.906	0.93	987.16	Strategy = Sorted;...
↳ Unknown (Incremental Sort)		148225	4.188187E+7	23409.08	20710.593	0.93	956.876	Parent Relationshi...
↳ Merge Join		148225	4.188187E+7	19096.0	10761.271	0.0	956.615	Parent Relationshi...
↳ Access (CTE Scan	table: t;	77000	249180.0	5197.5	1701.285	0.0	956.599	Parent Relationshi...
↳ Temporary (Matc		231000	85836212	5197.5	2004.698	0.0	0.002	Parent Relationshi...
↳ Access (CTE Scan	table: t;	231000	2073966.0	4620.0	188.992	0.0	0.0	Parent Relationshi...

Wnioski:

- zapytanie wykorzystujące funkcję okna (dense_rank), charakteryzuje się ponad 4-krotnie niższym kosztem (total cost)
- faktyczny czas wykonania zapytania jest ~50 razy krótszy (actual total time)
- wiersze w zapytaniu z inner-join musiały się zmaterializować, w rezultacie tworzonych jest 85 milionów wierszy pośrednich
- liczba wierszy jest mocno niedoszacowana względem faktycznej liczby wierszy

Alternatywne zapytanie korzystające z subquery nie wykonało się w rozsądnym czasie, nawet na ograniczonym zbiorze danych.

Porównanie planów zapytań - MSSQL:

• zapytanie

Z

dense_rank

Operation	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select	80598.5		21.091		BatchModeOnRowStoreUs...
Transformation (Gather Streams - Parallelism)	80598.5	3264	21.091	74.0	AvgRowSize = 55; Estimate...
Sort	80598.5	3264	20.5118	4.0	AvgRowSize = 55; Estimate...
Aggregate (Aggregate - Hash Match)	80598.5	3264	20.3601	2.0	AvgRowSize = 55; Estimate...
Filter	80598.5	3264	20.2202	0.0	AvgRowSize = 55; Estimate...
Value (Compute Scalar)	268662	270000	20.1879	2.0	AvgRowSize = 55; Estimate...
Unknown (Window Aggregate)	268662	270000	20.1873	0.0	AvgRowSize = 51; Estimate...
Sort	268662	270000	20.1866	23.0	AvgRowSize = 47; Estimate...
Value (Compute Scalar)	268662	270000	19.6296	3.0	AvgRowSize = 47; Estimate...
Full Index Scan (Clustered Index Scan)	ta...	270000	19.3517	36.0	AvgRowSize = 43; Estimate...

• zapytanie

Z

inner-join

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		11028.4		51.9911		BatchModeOnRowStoreUsed ...
Transformation (Gather Streams - Parallelism)		11028.4	3264	51.9911	6467.0	AvgRowSize = 51; EstimateCP...
Sort		11028.4	3264	51.888	8.0	AvgRowSize = 51; EstimateCP...
Value (Compute Scalar)		11028.4	3264	51.87	0.0	AvgRowSize = 51; EstimateCP...
Filter		11028.4	3264	51.87	0.0	AvgRowSize = 51; EstimateCP...
Value (Compute Scalar)		268661	270000	51.8654	0.0	AvgRowSize = 51; EstimateCP...
Aggregate (Aggregate - Hash Match)		268661	270000	51.8654	5929.0	AvgRowSize = 51; EstimateCP...
Hash Join (Left Outer Join - Hash)		2.30998E+6	49079870	51.052	311.0	AvgRowSize = 56; EstimateCP...
Aggregate (Aggregate - Hash Ma)		268661	270000	20.0224	97.0	AvgRowSize = 47; EstimateCP...
Value (Compute Scalar)		268662	270000	19.6296	3.0	AvgRowSize = 47; EstimateCP...
Full Index Scan (Clustered Index Scan)	table: [dbo].[...]	268662	270000	19.3517	24.0	AvgRowSize = 43; EstimateCP...
Aggregate (Aggregate - Hash Ma)		2.30998E+6	270000	22.7205	104.0	AvgRowSize = 24; EstimateCP...
Filter		2.31E+6	270000	19.3569	1.0	AvgRowSize = 64; EstimateCP...
Value (Compute Scalar)		2.31E+6	2310000	19.3569	20.0	AvgRowSize = 64; EstimateCP...
Full Index Scan (Clus table: [dbo].[...])		2.31E+6	2310000	19.3512	31.0	AvgRowSize = 47; EstimateCP...

Wnioski:

- dla ograniczonego zapytania, całkowity koszt zapytania z funkcją okna jest około 2,5 raza niższy względem zapytania z joinem, a faktyczny czas zapytania jest ~90 razy niższy.
- zapytanie jest zdecydowanie szybsze w porównaniu do Postgresa (~7 razy dla funkcji okna oraz ~4 razy dla zapytania z joinem)
- zapytanie na nieograniczonym zbiorze wykonało się w rozsądnym czasie zarówno dla zapytania z funkcją okna oraz joinem (~200 razy niższy całkowity koszt zapytania, zapytanie z funkcją okna wykonywało się około 400ms, zapytaniem z joinem około 1,5 minuty)

Porównanie wydajności i planów zapytań - SQLite:

• Z

funkcją

dense_rank:

Operation	Params	Raw Desc
Select		CO-ROUTINE t
Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
Full Scan	table: product_history;	SCAN product_history
Order By		USE TEMP B-TREE FOR ORDER BY
Full Scan	table: (subquery-3);	SCAN (subquery-3)
Unique		USE TEMP B-TREE FOR DISTINCT
Full Scan	table: t;	SCAN t
Order By		USE TEMP B-TREE FOR ORDER BY

• Z

inner-join:

Operation	Params	Raw Desc
Select		CO-ROUTINE dr
Operation (CO-ROUTINE)		MATERIALIZED t
Full Scan	table: product_history;	SCAN product_history
Unique		USE TEMP B-TREE FOR DISTINCT
Full Scan	table: p1;	SCAN p1
Unknown		BLOOM FILTER ON p2 (year=? AND productid=?)
Index Scan	table: p2; index: (year=?);	SEARCH p2 USING AUTOMATIC COVERING INDEX (year=? AND productid=?)
Group By		USE TEMP B-TREE FOR GROUP BY
Transformation (count(DISTINCT))		USE TEMP B-TREE FOR count(DISTINCT)
Full Scan	table: dr;	SCAN dr
Order By		USE TEMP B-TREE FOR ORDER BY

Wnioski:

- zapytanie korzystające z funkcji okna dla ograniczonego zbioru danych wykonuje się poniżej 1 sekundy, w przypadku zapytania z joinem czas wykonania zapytania wynosi około 1 minuty (nie są to jednak dokładne wyniki ze względu na paginację, a SQLite nie posiada opcji explain analize z dokładną informacją odnośnie faktycznego czasu wykonania zapytania)
 - w planie zapytania z inner-joinem widzimy, że dane muszą się faktycznie zmaterializować
-

Zadanie 3

Funkcje lag(), lead()

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje lag(), lead()

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid
order by date) as previousprodprice,
       lead(unitprice) over (partition by productid
order by date) as nextprodprice
       from product_history
       )
select * from t
where productid = 1 and year(date) = 2022
order by date;
```

Jak działają funkcje lag(), lead()?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

```
-- ...
```

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta, nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

-- ...

Zadanie 5

Funkcje `first_value()`, `last_value()`

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `first_value()`, `last_value()`.

Skomentuj uzyskane wyniki. Czy funkcja `first_value` pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja `last_value()` pokazuje najtańszy produkt?

Co jest przyczyną takiego działania funkcji `last_value`.

Co trzeba zmienić żeby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

```
select productid, productname, unitprice, categoryid,  
       first_value(productname) over (partition by categoryid  
order by unitprice desc) first,  
       last_value(productname) over (partition by categoryid  
order by unitprice desc) last  
from products  
order by categoryid, unitprice desc;
```

Wyniki:

productid	productname	unitprice	categoryid	first	last
38	Côte de Blaye	263.5	1	Côte de Blaye	Côte de Blaye
43	Iphoh Coffee	46	1	Côte de Blaye	Iphoh Coffee
2	Chang	19	1	Côte de Blaye	Chang
1	Chai	18	1	Côte de Blaye	Chartreuse verte
76	Lakkalikööri	18	1	Côte de Blaye	Chartreuse verte
35	Steeleye Stout	18	1	Côte de Blaye	Chartreuse verte
39	Chartreuse verte	18	1	Côte de Blaye	Chartreuse verte
70	Outback Lager	15	1	Côte de Blaye	Outback Lager
34	Sasquatch Ale	14	1	Côte de Blaye	Laughing Lumberjack Lager
67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Laughing Lumberjack Lager
75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Rhönbräu Klosterbier
24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
63	Vegie-spread	43.9	2	Vegie-spread	Vegie-spread

Figure 3: alt text

Funkcja `first_value()` zwraca pierwszą wartość w danym oknie, funkcja `last_value()` zwraca ostatnią wartość w danym oknie, przy czym zakres funkcji `last_value` jest od początku do **aktualnego wiersza** (rows between unbounded preceding and current row). W naszym przypadku, funkcja `first_value()` faktycznie będzie pokazywała najdroższy produkt w danej kategorii, natomiast funkcja `last_value()` będzie zawsze pokazywała produkt z danego wiersza (bo on jest najtańszy licząc od początku do aktualnego wiersza). W celu wykorzystania funkcji `last_value` do wskazywania najtańszego produktu w danej kategorii, musimy zmienić zakres tej funkcji:

```
-- wcześniej, zakres do aktualnego wiersza
last_value(productname) over (partition by categoryid
    order by unitprice desc) last

-- poprawiona wersja, zakres całego okna
last_value(productname) over (partition by categoryid
    order by unitprice desc rows between unbounded preceding and unbounded
    following) last
```

Rezultaty poprawionego zapytania:

	productid	productname	unitprice	categoryid	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
2	43	Iphoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
3	2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
4	1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
5	76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
6	35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
7	39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
8	70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
9	34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Guaraná Fantástica
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Guaraná Fantástica
12	24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
13	63	Vegie-spread	43.9	2	Vegie-spread	Aniseed Syrup

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

-- ...

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

Zapytanie realizujące opisane zadanie w Postgresie:

```

-- z funkcją okna
with t as (select od.orderid                                as id,
                  od.productid,
                  date_part('month', o.orderdate)           as month,
                  o.orderdate                               as date,
                  od.unitprice * od.quantity * (1 - od.discount) as total,
                  sum(od.unitprice * od.quantity * (1 - od.discount)) over (
                    partition by od.productid,
                               date_part('year', o.orderdate),
                               date_part('month', o.orderdate)
                    order by od.productid, o.orderdate
                  )                                           as cum_total
            from orderdetails od
            join orders o on od.orderid = o.orderid)
select t.id,
       t.productid,
       t.month,
       t.date,
       t.total,
       t.cum_total
from t;

-- bez funkcji okna (inner-join)
with t as (select od.orderid                                as id,
                  od.productid,
                  date_part('month', o.orderdate)           as month,
                  o.orderdate                               as date,
                  od.unitprice * od.quantity * (1 - od.discount) as total
            from orderdetails od
            join orders o on od.orderid = o.orderid)
select t.id,
       t.productid,
       t.month,
       t.date,
       t.total,
       sum(t2.total) as cum_total
from t
     left join t t2
       on t2.productid = t.productid
        and date_part('year', t2.date) = date_part('year', t.date)
        and date_part('month', t2.date) = date_part('month',
t.date)
        and (
            t2.date < t.date
            or (t2.date = t.date and t2.id <= t.id)
        )
group by t.id,
         t.productid,
         t.month,

```

```

t.date,
t.total
order by t.productid,
t.date;

```

Rezultaty zapytania:

• z funkcją okna:

	id	productid	month	date	total	cum_total
1	10285	1	8	1996-08-20	595.2299861669541	595.2299861669541
2	10294	1	8	1996-08-30	406.45998764038086	1001.6899738073349
3	10317	1	9	1996-09-30	300.68999195098877	300.68999195098877
4	10348	1	11	1996-11-07	184.37999382019046	184.37999382019046
5	10354	1	11	1996-11-14	226.59999465942383	410.9799884796143
6	10370	1	12	1996-12-03	184.76999380588535	184.76999380588535
7	10406	1	1	1997-01-07	252.03999710083008	252.03999710083008
8	10413	1	1	1997-01-14	441.25999450683594	693.299991607666
9	10477	1	3	1997-03-17	229.01999473571777	229.01999473571777
10	10522	1	4	1997-04-30	621.3299996852875	621.3299996852875
11	10526	1	5	1997-05-05	180.98999294281	180.98999294281
12	10576	1	6	1997-06-23	198.55999946594238	198.55999946594238
13	10590	1	7	1997-07-07	404.770004577637	404.770004577637
14	10609	1	7	1997-07-24	55.8500002384186	460.62000048160553
15	10611	1	7	1997-07-25	188.6500015258789	649.2700020074844
16	10628	1	8	1997-08-12	480.36000061035156	480.36000061035156
17	10646	1	8	1997-08-27	344.8300018310547	825.1900024414062
18	10689	1	10	1997-10-01	485.92000007629395	485.92000007629395

• z inner-joinem:

	id	productid	month	date	total	cum_total
1	10285	1	8	1996-08-20	595.2299861669541	595.2299861669541
2	10294	1	8	1996-08-30	406.45998764038086	1001.6899738073349
3	10317	1	9	1996-09-30	300.68999195098877	300.68999195098877
4	10348	1	11	1996-11-07	184.37999382019046	184.37999382019046
5	10354	1	11	1996-11-14	226.59999465942383	410.9799884796143
6	10370	1	12	1996-12-03	184.76999380588535	184.76999380588535
7	10406	1	1	1997-01-07	252.03999710083008	252.03999710083008
8	10413	1	1	1997-01-14	441.25999450683594	693.299991607666
9	10477	1	3	1997-03-17	229.01999473571777	229.01999473571777
10	10522	1	4	1997-04-30	621.3299996852875	621.3299996852875
11	10526	1	5	1997-05-05	180.98999294281	180.98999294281
12	10576	1	6	1997-06-23	198.55999946594238	198.55999946594238
13	10590	1	7	1997-07-07	404.770004577637	404.770004577637
14	10609	1	7	1997-07-24	55.8500002384186	460.62000048160553
15	10611	1	7	1997-07-25	188.6500015258789	649.2700020074844
16	10628	1	8	1997-08-12	480.36000061035156	480.36000061035156
17	10646	1	8	1997-08-27	344.8300018310547	825.1900024414062
18	10689	1	10	1997-10-01	485.92000007629395	485.92000007629395

Jak widać na załączonych zrzutach ekranu, wartości te są kumulowane w ramach danego miesiąca. Wyniki są identyczny w przypadku obu zapytań.

Porównanie wydajności i planów zapytań - Postgres:

- zapytanie z funkcją okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↳ Select								Planning = ("Shar...
↳ Access (Subquery Scan)		2155	2155.0	369.02	3.036	212.78	2.091	Parallel Aware = f...
↳ Transformation (WindowAgg)		2155	2155.0	347.47	2.906	212.78	2.09	Parent Relationsh...
↳ Sort		2155	2155.0	218.17	2.192	212.78	2.053	Parent Relationsh...
↳ Hash Join		2155	2155.0	93.47	1.086	32.68	0.434	Parent Relationsh...
↳ Full Scan (Seq S table: orderdetails;)		2155	2155.0	33.55	0.214	0.0	0.038	Parent Relationsh...
↳ Transformation (H)		830	830.0	22.3	0.331	22.3	0.327	Parent Relationsh...
↳ Full Scan (Se table: orders;)		830	830.0	22.3	0.202	0.0	0.017	Parent Relationsh...

- zapytanie z inner-joinem:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
↳ Select								Planning = ("Shared ...
↳ Sort		1246	2155.0	675.76	7.112	672.64	7.043	Parallel Aware = fals...
↳ Hash Join		2155	2155.0	109.83	1.321	32.68	0.712	Parent Relationship ...
↳ Full Scan (Seq Scan) table: orderdetails;)		2155	2155.0	33.55	0.284	0.0	0.105	Parent Relationship ...
↳ Transformation (Hash)		830	830.0	22.3	0.521	22.3	0.52	Parent Relationship ...
↳ Full Scan (Seq Scan table: orders;)		830	830.0	22.3	0.322	0.0	0.03	Parent Relationship ...
↳ Aggregate		1246	2155.0	498.95	6.291	486.49	6.092	Strategy = Hashed; ...
↳ Merge Join		2155	3851.0	454.17	5.319	324.83	4.005	Parent Relationship ...
↳ Sort		2155	2155.0	167.8	3.157	162.42	3.052	Parent Relationship ...
↳ Access (CTE Sc table: t;)		2155	2155.0	43.1	2.141	0.0	0.717	Parent Relationship ...
↳ Sort		2155	5546.0	167.8	1.216	162.42	0.94	Parent Relationship ...
↳ Access (CTE Sc table: t;)		2155	2155.0	43.1	0.322	0.0	0.005	Parent Relationship ...

Wnioski:

- zapytanie z funkcją okna charakteryzuje się około 2 razy niższym kosztem zapytania
- ze względu na mały rozmiar danych (tabele orders i ordershistory), zapytania są generalnie bardzo szybkie (~30ms), więc nie ma sensu porównywać bezpośrednio czasu zapytań

Zapytania dla MSSQL oraz SQLite są identyczne, z dokładnością do funkcji ekstrahującej miesiąc oraz rok z daty.

Porównanie wydajności i planów zapytań - MSSQL:

- z zapytaniem z funkcją okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
↳ Select		2155		0.103945		CardinalityEstima...
↳ Value (Compute Scalar)		2155		0.103945		AvgRowSize = 34; Es
↳ Value (Compute Scalar)		2155		0.103729		AvgRowSize = 50; Es
↳ Aggregate (Aggregate - Stream Aggrega		2155	2155	0.103514	40.0	AvgRowSize = 75; Es
↳ Temporary (Window Spool)		4310	4310	0.10216	36.0	AvgRowSize = 75; Es
↳ Transformation (Segment)		2155	2155	0.0968824	11.0	AvgRowSize = 75; Es
↳ Transformation (Segment)		2155	2155	0.0968824	10.0	AvgRowSize = 67; Es
↳ Sort		2155	2155	0.096747	10.0	AvgRowSize = 59; Es
↳ Merge Join (Inner Join)		2155	2155	0.0481589	3.0	AvgRowSize = 59; Es
↳ Value (Compute Sc		830		0.021315		AvgRowSize = 34; Es
↳ Full Index Scan i table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	0.0	AvgRowSize = 22; Es
↳ Value (Compute Sc		2155		0.0147569		AvgRowSize = 35; Es
↳ Full Index Scan i table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	1.0	AvgRowSize = 31; Es

• Z

inner-join:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
↳ Select		2155		0.363612		CardinalityEstimation...
↳ Sort		2155	2155	0.363612	34.0	AvgRowSize = 34; Es...
↳ Value (Compute Scalar)		2155		0.315023		AvgRowSize = 34; Es...
↳ Aggregate (Aggregate - Stream Aggrega		2155	2155	0.315023	32.0	AvgRowSize = 34; Es...
↳ Sort		2155	3851	0.312653	31.0	AvgRowSize = 30; Es...
↳ Value (Compute Scalar)		2155		0.264065		AvgRowSize = 30; Es...
↳ Hash Join (Right Outer Join -		2155	3851	0.263849	28.0	AvgRowSize = 34; Es...
↳ Value (Compute Scalar)		2155		0.0798844		AvgRowSize = 30; Es...
↳ Hash Join (Inner Join -		2155	2155	0.0796689	15.0	AvgRowSize = 34; Es...
↳ Value (Compute Sc		830		0.021315		AvgRowSize = 26; Es...
↳ Full Index Scan (table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	2.0	AvgRowSize = 22; Es...
↳ Value (Compute Sc		2155		0.0147569		AvgRowSize = 19; Es...
↳ Full Index Scan (table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	9.0	AvgRowSize = 31; Es...
↳ Hash Join (Inner Join - Hi		2155	2155	0.0796689	2.0	AvgRowSize = 34; Es...
↳ Value (Compute Scala		830		0.021315		AvgRowSize = 26; Es...
↳ Full Index Scan (table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	0.0	AvgRowSize = 22; Es...
↳ Value (Compute Scala		2155		0.0147569		AvgRowSize = 19; Es...
↳ Full Index Scan (table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	1.0	AvgRowSize = 31; Es...

Wnioski:

- koszt całkowity dla funkcji okna jest ~3 krotnie niższy, natomiast czas wykonania jest bardzo zbliżony (ze względu na mały ilość danych)

Porównanie wydajności i planów zapytań - SQLite:

• Z

funkcją

okna:

Operation	Params	Raw Desc
↳ Select		CO-ROUTINE t
↳ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
↳ Index Scan	table: od; index: ProductsOrder_Details;	SCAN od USING INDEX ProductsOrder_Details
↳ Index Scan	table: o;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
Transformation (LAST 4 TERMS OF ORDER BY)		USE TEMP B-TREE FOR LAST 4 TERMS OF ORDER BY
↳ Full Scan	table: (subquery-3);	SCAN (subquery-3)
↳ Full Scan	table: t;	SCAN t

• Z

inner-join:

Operation	Params	Raw Desc
↳ Select		MATERIALIZED t
↳ Operation (MATERIALIZED)		SCAN od
↳ Full Scan	table: od;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
↳ Index Scan	table: o;	SCAN od USING INDEX sqlite_autoindex_Order_Details_1
↳ Index Scan	table: od; index: sqlite_autoindex_Order;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
↳ Index Scan	table: o;	BLOOM FILTER ON t2 (productid=? AND month=?)
Unknown		SEARCH t2 USING AUTOMATIC COVERING INDEX (productid=? ..
↳ Index Scan	table: t2; index: (productid=?;	USE TEMP B-TREE FOR ORDER BY
Order By		

Wnioski:

- czas wykonania zapytań jest bardzo zbliżony (~400ms) zarówno dla funkcji okna i inner-joina, jest to około 20-krotnie dłużej niż to samo zapytanie w Postgres i MSSQL
- ze względu na format daty w tabelach orders i orderhistory konieczne było niewygodne castowanie daty do odpowiedniego formatu (cast(substr(o.orderdate, 1, instr(o.orderdate, '/') - 1) as int) as month)

Zadanie 8

Wykonaj kilka “własnych” przykładowych analiz.

Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)?
Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

-- ...

Punktacja

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2
razem	11